CSA1712

ARTIFICIAL INTELLIGENCE

NAME: VARSHINI M

REG NO.: 192511215

DESIGN TASK 3

AI-BASED FRAUD DETECTION FRAMEWORK

1. Problem Understanding and Requirement Analysis

Financial institutions process millions of transactions every day through credit cards, debit cards, UPI, online banking, and digital wallets. Among these transactions, a small percentage may be fraudulent.

Traditional rule-based fraud detection systems depend on predefined conditions such as:

- Unusually large transaction amount
- Multiple transactions within a short period
- Transactions from unusual locations
- Repeated failed transactions
- Unexpected changes in spending behaviour

However, fraud patterns continuously change. Therefore, a scalable AI-based system is required that can process large volumes of financial transactions, identify unusual behaviour, and continuously improve fraud detection.

Objective

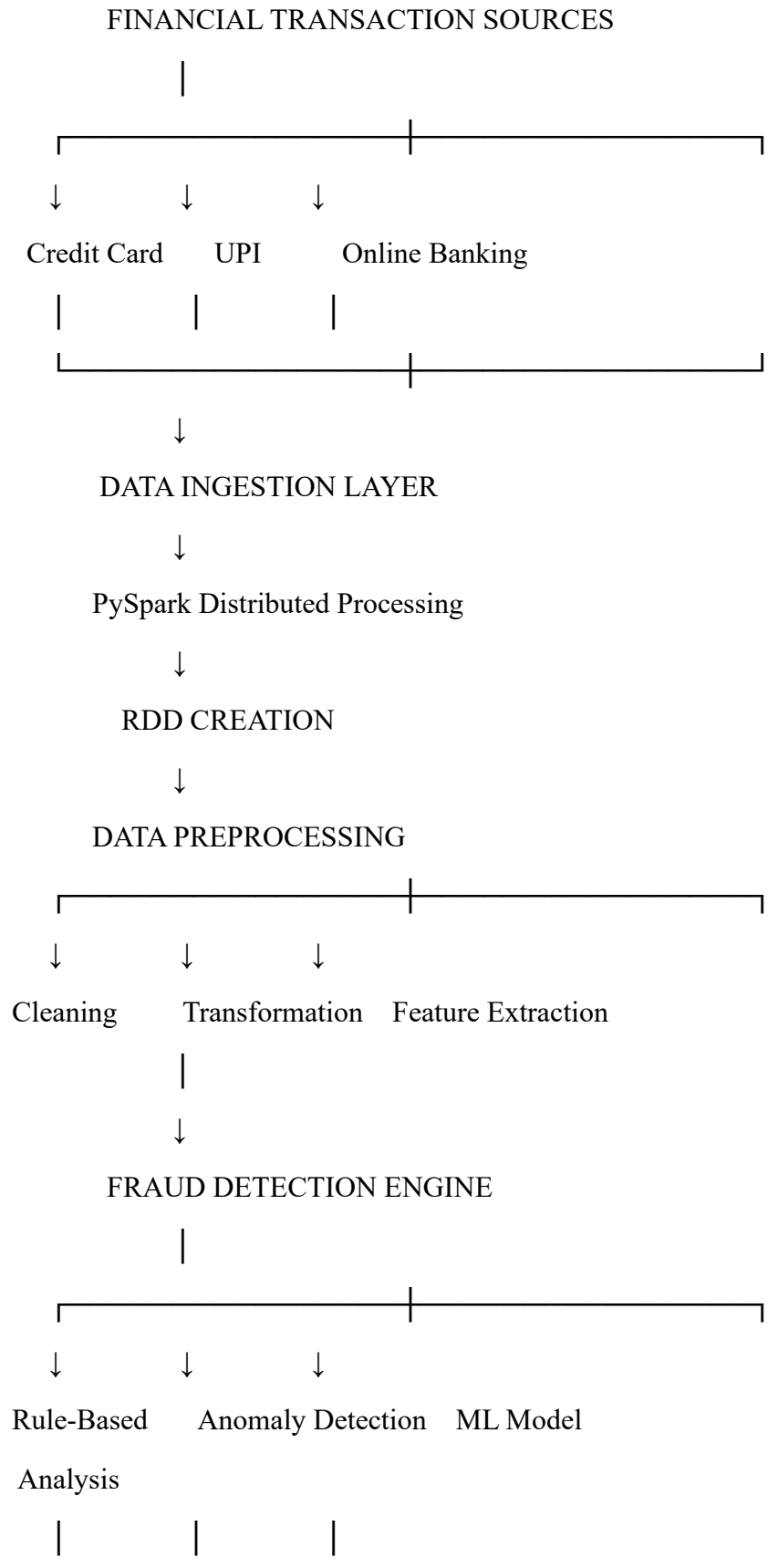
The objective is to design a **distributed AI-based fraud detection framework using PySpark RDDs and intelligent agents** that can:

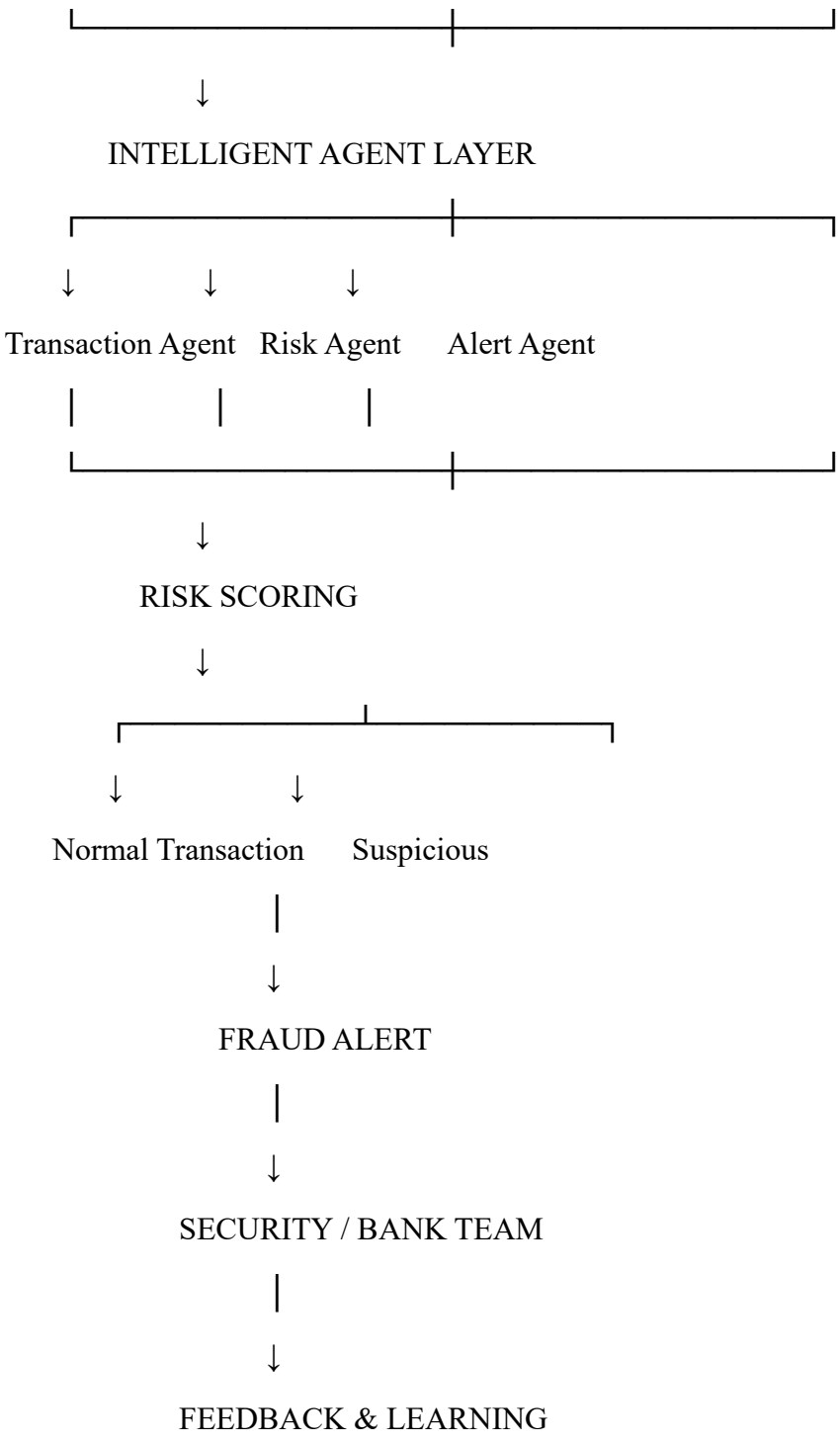
1. Process large-scale financial transaction data.
2. Detect abnormal and suspicious transactions.
3. Apply machine-learning/anomaly-detection techniques.
4. Generate real-time fraud alerts.
5. Allow intelligent agents to collaborate.
6. Continuously update fraud patterns based on new transaction behaviour.
7. Scale efficiently when transaction volume increases.

---

2. Proposed System Architecture

The proposed system consists of multiple layers.





---

3. Data Flow and Processing Pipeline

The complete data pipeline is designed to process financial transactions in a distributed environment.

Step 1: Data Collection

Transaction data can contain:

Feature	Description
Transaction ID	Unique transaction identifier
User ID	Customer identifier
Amount	Transaction amount
Timestamp	Date and time
Location	Transaction location
Merchant ID	Merchant identifier
Payment Method	Card, UPI, wallet, etc.
Device ID	Device used
Previous Transaction	Previous customer activity
Fraud Label	Fraudulent/legitimate

Step 2: RDD Creation

PySpark is used to distribute the transaction data across multiple nodes.

Example:

```
transactions = sc.textFile("transactions.csv")


transaction_rdd = transactions.map(
    lambda x: x.split(",")
)
```

Each transaction becomes an element of the distributed RDD.

---

4. PySpark RDD Operations

RDD operations are divided into **transformations** and **actions**.

Transformations

Transformations create a new RDD from an existing RDD.

Map

Used to convert raw transaction records into structured records.

```
mapped_rdd = transaction_rdd.map(
    lambda x: (x[1], float(x[2]))
)
```

Filter

Used to remove invalid transactions.

```
valid_rdd = mapped_rdd.filter(
    lambda x: x[1] > 0
)
```

ReduceByKey

Used to calculate the total transaction amount for each customer.

```
total_amount = mapped_rdd.reduceByKey(
    lambda a, b: a + b
)
```

GroupByKey

Can be used to group transactions belonging to the same customer.

```
customer_transactions = transaction_rdd.groupByKey()
```

---

5. Fraud Detection Features

The system extracts behavioural features from transaction data.

Transaction Amount

A very high transaction compared with the customer's normal spending pattern may indicate fraud.

Transaction Frequency

A sudden increase in the number of transactions within a short period can indicate suspicious activity.

Location Change

A transaction occurring in a geographically unusual location can increase the risk score.

Device Change

A transaction from an unknown device can be considered suspicious.

Time-Based Behaviour

Transactions occurring at unusual times can contribute to the risk score.

Spending Pattern

The system compares the current transaction with the user's historical behaviour.

---

6. Anomaly Detection

The framework uses anomaly detection to identify transactions that differ significantly from normal behaviour.

A simple anomaly score can be represented as:

$$
A = w_1X_1+w_2X_2+w_3X_3+w_4X_4+w_5X_5
$$

where:

- X_1 = amount anomaly
- X_2 = frequency anomaly
- X_3 = location anomaly
- X_4 = device anomaly
- X_5 = behavioural anomaly
- $w_1, w_2, ... w_5$ = feature weights

The final value is converted into a **fraud risk score**.

---

7. Risk Classification

The system classifies transactions according to their risk score.

Risk Score	Classification	Action
0–30	Low Risk	Allow transaction
31–60	Medium Risk	Additional verification
61–80	High Risk	Temporary review
81–100	Critical Risk	Generate fraud alert

This approach avoids treating every unusual transaction as definite fraud.

---

8. Intelligent Agent Integration

The system uses multiple intelligent agents that specialize in different tasks.

8.1 Transaction Monitoring Agent

This agent continuously monitors incoming transactions.

Responsibilities

- Observe new transactions.
- Extract transaction information.
- Identify unusual activities.
- Forward suspicious transactions to the risk agent.

8.2 Risk Assessment Agent

This agent calculates the fraud risk score.

It considers:

- Transaction amount
- Location
- Device
- Transaction frequency
- Historical behaviour
- Anomaly score

It then classifies the transaction as low, medium, high, or critical risk.

8.3 Pattern Detection Agent

This agent searches for recurring fraud patterns.

For example:

Multiple accounts

↓

Same device

↓

Similar transactions

↓

Unusual locations

↓

Possible coordinated fraud

8.4 Alert Agent

The alert agent responds to high-risk transactions.

It can:

- Generate fraud alerts.
- Notify security teams.
- Request additional authentication.
- Temporarily flag transactions.
- Record suspicious activity.

8.5 Learning Agent

The learning agent receives feedback from confirmed fraud investigations.

Transaction

↓

Fraud Prediction

↓

Human Investigation

↓

Confirmed Fraud / Legitimate

↓

Learning Agent

↓

Update Detection Patterns

This allows the system to adapt to changing fraud techniques.

---

9. Multi-Agent Collaboration

The agents communicate with one another rather than working independently.

Transaction Agent

↓

Pattern Agent

↓

Risk Agent

↓

Alert Agent

↓

Learning Agent

↓

Updated Fraud Patterns

For example, when a transaction is detected from an unfamiliar device:

1. The Transaction Agent detects the new device.
2. The Pattern Agent compares it with previous behaviour.
3. The Risk Agent calculates the risk score.
4. The Alert Agent generates an alert if the score is high.
5. The Learning Agent uses the final investigation result to improve future detection.

This collaborative architecture provides stronger fraud detection than a single rule-based system.

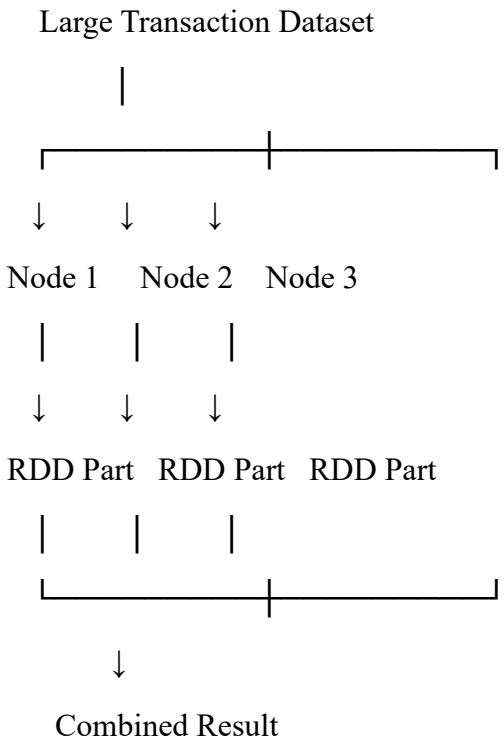
---

10. Scalability and Distributed Computing

Financial institutions may process millions of transactions every day. Processing all data on a single machine can create:

- High processing time
- Memory limitations
- System bottlenecks
- Poor scalability

PySpark addresses this problem through distributed computing.



RDD partitions allow different parts of the dataset to be processed in parallel.

Scalability Strategy

When transaction volume increases, additional worker nodes can be added to the Spark cluster.

Therefore:

[
More\ Data \rightarrow More\ Partitions \rightarrow More\ Parallel\ Processing
]

This makes the framework suitable for large-scale financial applications.

---

11. Fault Tolerance

PySpark RDDs provide fault tolerance through **lineage information**.

If a partition is lost, Spark can reconstruct the required data using the transformation history.

For example:

Raw Data

↓

Map

↓

Filter

↓

Feature Extraction

↓

Risk Calculation

If an intermediate partition fails, the required portion can be recomputed.

This improves system reliability.

---

12. Real-Time Fraud Detection

For real-time applications, new transaction streams can continuously enter the system.

New Transaction

↓

Feature Extraction

↓

Anomaly Detection

↓

Risk Score

↓

Agent Decision

↓

Alert / Approve

For example:

Transaction: ₹1,50,000

User's normal transaction: ₹2,000–₹10,000

Location: New country

Device: Unknown

Transaction frequency: 5 transactions within 2 minutes

The system may calculate a high risk score and generate:

HIGH-RISK TRANSACTION – Additional Verification Required

---

13. Innovation and Creativity

The proposed framework combines three major technologies:

1. Distributed Computing

PySpark enables processing of large transaction datasets.

2. Artificial Intelligence

Anomaly detection and machine-learning models identify suspicious behaviour.

3. Multi-Agent Intelligence

Specialized agents collaborate to monitor, assess, alert, and learn from fraud events.

The major innovation is the **continuous feedback loop**:

Detect

↓

Analyze

↓

Alert

↓

Investigate

↓

Receive Feedback

↓

Learn

↓

Improve Detection

This allows the system to adapt to new fraud patterns instead of depending entirely on fixed rules.

---

14. Security and Privacy

Since financial transaction data is sensitive, the system should implement:

- Encryption of sensitive data

- Secure authentication
- Role-based access control
- Data anonymization
- Secure API communication
- Audit logging
- Minimal access to customer information

Only authorized personnel should be able to view detailed transaction information.

---

15. System Output

The system can generate a fraud monitoring dashboard containing:

Transaction ID	Amount	Risk Score	Risk Level	Action
T1001	₹2,500	12	Low	Approved
T1002	₹18,000	45	Medium	Verify
T1003	₹75,000	78	High	Review
T1004	₹1,50,000	94	Critical	Alert

This allows security teams to quickly identify high-risk transactions.

---

16. Expected Results

The proposed framework is expected to:

1. Process large transaction datasets efficiently.
2. Reduce the time required for fraud detection.
3. Identify unusual transaction patterns.
4. Generate real-time risk scores.
5. Reduce dependency on static rules.
6. Improve adaptability to new fraud patterns.
7. Support large-scale distributed processing.
8. Provide explainable reasons for high-risk alerts.

---

17. Evaluation Metrics

The system can be evaluated using:

Accuracy

$$
Accuracy=\frac{TP+TN}{TP+TN+FP+FN}
$$

Precision

$$
Precision=\frac{TP}{TP+FP}
$$

Recall

$$
Recall=\frac{TP}{TP+FN}
$$

F1-Score

$$
F1=2\times\frac{Precision\times Recall}{Precision+Recall}
$$

For fraud detection, **precision and recall are particularly important**.

A system with very high accuracy can still be ineffective if it fails to detect actual fraudulent transactions.

---

18. Challenges and Limitations

The proposed system has several challenges.

Data Imbalance

Fraudulent transactions are usually much fewer than legitimate transactions.

False Positives

Legitimate transactions may sometimes be classified as suspicious.

Evolving Fraud Patterns

Fraudsters continuously change their strategies.

Privacy

Financial data requires strong security and privacy controls.

Model Drift

The performance of a model may decrease as transaction behaviour changes.

Computational Cost

Processing extremely large transaction streams requires sufficient computing resources.

---

19. Future Enhancements

Future versions can include:

- Deep learning-based anomaly detection
- Graph Neural Networks for detecting fraud networks
- Explainable AI using SHAP
- Real-time streaming with Spark Structured Streaming
- Federated learning for privacy-preserving model training
- Adaptive reinforcement learning
- Blockchain-based transaction verification
- Automated threat intelligence integration

---

20. Conclusion

The proposed **AI-Based Fraud Detection Framework** combines **PySpark RDD-based distributed processing, anomaly detection, machine learning, and intelligent multi-agent collaboration** to create a scalable fraud detection system.

PySpark enables large volumes of financial transactions to be processed in parallel, while anomaly detection identifies transactions that differ from normal customer behaviour. Intelligent agents divide the overall responsibility into specialized tasks such as transaction monitoring, risk assessment, pattern detection, alert generation, and continuous learning.

The feedback mechanism allows the system to learn from confirmed fraud cases and adapt to emerging fraud patterns. The architecture is therefore more flexible than a traditional rule-based system.

Overall, the proposed solution is **scalable, practical, innovative, and suitable for large-scale financial environments**, while privacy, false positives, model bias, and human verification must be carefully addressed before real-world deployment.

RUBRIC ALIGNMENT

Evaluation Criterion	How This Design Achieves Excellent (5)
Problem Understanding & Requirement Analysis – 10%	Clearly defines financial fraud problem, objectives, constraints, and requirements
System Architecture Design – 20%	Provides complete layered architecture with data flow and intelligent-agent interaction
Use of PySpark & RDD Operations – 15%	Includes map(), filter(), reduceByKey(), groupByKey(), partitioning and fault tolerance
Intelligent Agent Integration – 15%	Uses Transaction, Risk, Pattern, Alert and Learning Agents with collaborative decision-making
Scalability & Distributed Computing – 10%	Explains RDD partitioning, parallel processing, worker nodes and fault tolerance
Data Flow & Processing Pipeline – 10%	Provides a clear end-to-end transaction-to-alert pipeline
Innovation & Creativity – 10%	Combines distributed AI, multi-agent collaboration and continuous feedback learning
Clarity, Presentation & Documentation – 10%	Includes structured sections, tables, equations, architecture diagrams and implementation examples
Overall Strength	